RED TEAM 27

Capstone Security Assessment

Test 7: Credential Stuffing & Bot Detection Assessment

Brentwood Bank — Online Banking Portal

https://secure.myvirtualbranch.com/brentwoodbank/

Test conducted by

Pragya Mittal

Client Liaison & Security Architect

MSISPM Capstone — Carnegie Mellon University

April 10, 2026

CONFIDENTIAL

1. Executive Summary

This report documents the findings from Test 7 of the Red Team 27 capstone security assessment of Brentwood Bank's online banking portal. The test evaluated the portal's controls against automated credential stuffing attacks — specifically whether the portal can detect, rate-limit, or block repeated automated login attempts from a single source, and whether the step-up authentication mechanism (CAPTCHA) triggered as a result is sufficiently robust to deter a determined attacker.

The test was conducted using a Python-based Playwright browser automation script routed through an OWASP ZAP proxy for full traffic capture. The script submitted fake credentials against the live login endpoint using a real browser instance to bypass the portal's F5 BIG-IP bot defense JavaScript challenge. All testing was performed under an authorized Rules of Engagement (ROE) document signed by Brentwood Bank.

Key findings: The portal's F5 BIG-IP layer provides meaningful bot detection for raw HTTP clients (e.g., curl, requests) by serving a JavaScript challenge that non-browser clients cannot pass. However, browser-based automation tools (Playwright, Selenium) can pass this challenge, reducing its effectiveness as a bot control. The portal triggered a distorted-text CAPTCHA challenge after a small number of automated login attempts. While the CAPTCHA presence is a positive control, distorted-text CAPTCHAs are considered a weak form of step-up authentication and are bypassable by modern OCR and machine learning tools. Additionally, the CAPTCHA functions as a soft block only — upon completion, users are returned to the login page with no lockout, delay, or escalation, allowing a determined attacker to continue credential stuffing indefinitely.

2. Test Environment

Parameter	Value
Date	April 10, 2026
Tester	Pragya Mittal — Client Liaison & Security Architect
Test Device	Dell Inc. Dell 14 Plus DB14250 (32 GB RAM)
OS	Ubuntu 24.04.4 LTS
Automation Tool	Python 3.12 + Playwright (Chromium)
Proxy / Capture Tool	OWASP ZAP (Proxy mode, localhost:8080)
Target URL	https://secure.myvirtualbranch.com/brentwoodbank/signin.aspx
Bot Defense Layer	F5 BIG-IP (TSPD JavaScript challenge)
Test Credentials	Fake usernames/passwords (testuser001–fakeuser010) + authorized test account

3. Scope & Objectives

3.1 Scope

This test was scoped to the authenticated login endpoint of the online banking portal at secure.myvirtualbranch.com/brentwoodbank/signin.aspx. Testing was limited to automated login attempts using non-existent fake credentials, with one authorized test account used as a functional baseline. No real customer credentials were used or targeted. All testing was performed under a signed ROE authorizing credential stuffing and bot detection testing.

3.2 Objectives

This test was designed to determine:

- Whether the portal's bot defense layer (F5 BIG-IP) can detect and block automated login attempts from browser-based automation tools.
- How many automated login attempts are permitted before a step-up authentication challenge (CAPTCHA) is triggered.
- Whether the CAPTCHA mechanism is sufficiently robust to deter a determined attacker, or whether it can be bypassed by automation.
- Whether triggering the CAPTCHA results in a hard lockout or only a soft challenge that allows continued attempts after completion.
- Whether the overall login flow conforms to FFIEC guidance and industry best practices for bot and credential stuffing controls on online banking portals.

4. Method

4.1 Test Name

Credential Stuffing & Bot Detection Assessment

4.2 Background: F5 BIG-IP Bot Defense Discovery

Initial testing using Python's requests library revealed that the portal is protected by an F5 BIG-IP bot defense layer. Rather than serving the login form directly, the portal returned a JavaScript challenge page (identifiable by TSPD cookies and bobcmn/failureConfig JavaScript variables) to non-browser HTTP clients. This challenge executes browser fingerprinting logic including screen resolution, timezone, plugin enumeration, and behavioral signals that a headless HTTP client cannot replicate.

This finding prompted the switch to Playwright, a browser automation framework that launches a real Chromium browser instance and can pass the F5 JavaScript challenge. All subsequent testing was performed via Playwright routed through OWASP ZAP.

4.3 Test Steps

Step 1 — Environment Setup: OWASP ZAP was launched and configured as an HTTPS proxy on localhost:8080. Playwright was configured to launch a Chromium browser instance routed through ZAP with SSL verification disabled to allow ZAP to intercept HTTPS traffic.

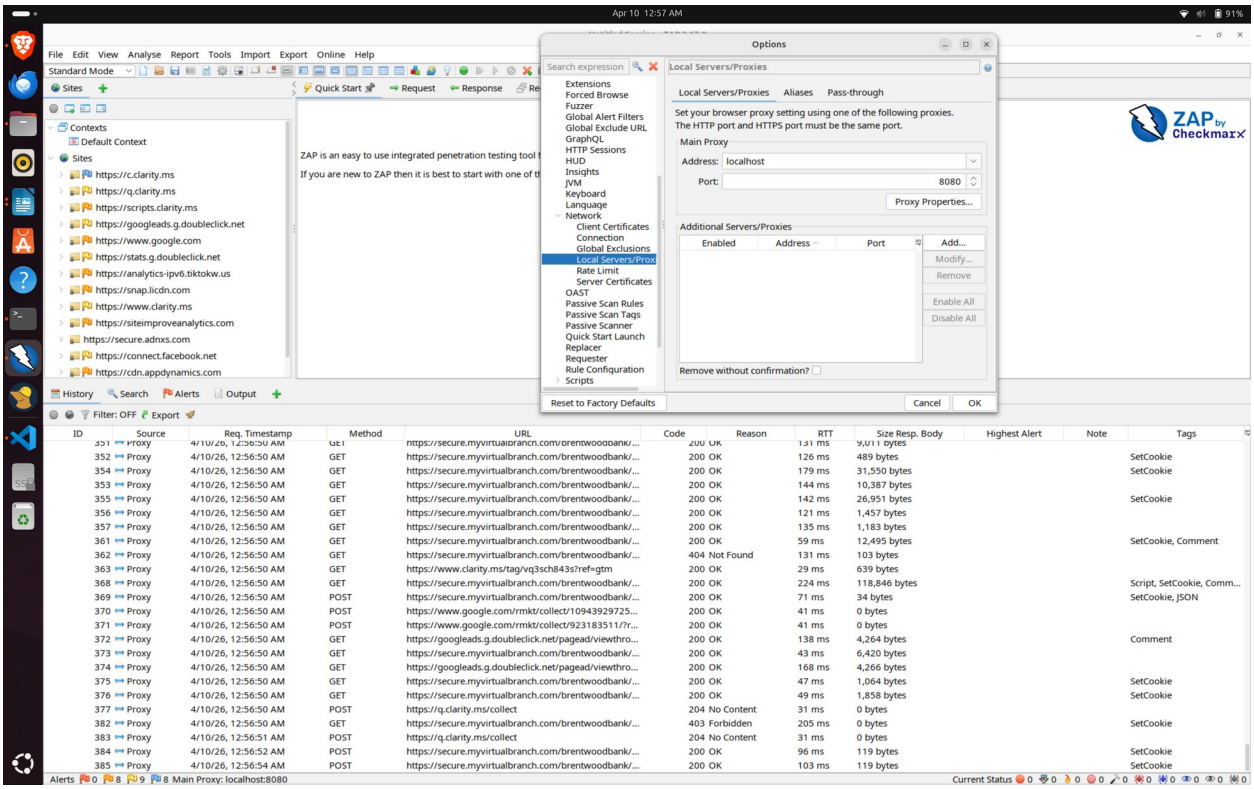

Figure 1: OWASP ZAP proxy configured and running on localhost:8080

Step 2 — Baseline Login Verification: The authorized test account (balluff212) was used to confirm that a successful login via the automation script correctly navigated through PostLogin.aspx and resolved to the authenticated accounts page. This established a clean baseline and confirmed the script's classification logic was functioning correctly.

```
(pentest-env) pragyamittal221@peggy:~/Downloads$ python3 brentwood_bot_test.py
[*] Target:      https://secure.myvirtualbranch.com/brentwoodbank/signin.aspx
[*] Proxy:       ZAP ON  (http://127.0.0.1:8080)
[*] Credentials: 1 pairs loaded
[*] Delay range: 0.5–2.0s between attempts
[*] Log file:    results_20260410_010916.csv

[*] SSL verification disabled for ZAP proxy — warnings suppressed

    [debug] final URL: https://secure.myvirtualbranch.com/brentwoodbank/React/Accounts.aspx

[001] balluff212                → LOGIN_SUCCESS

  Attempts made:     1
  Detection:        not triggered
  Elapsed time:      23.2s
  Results saved to:  results_20260410_010916.csv

(pentest-env) pragyamittal221@peggy:~/Downloads$
```

Figure 2: Successful baseline login confirmed via automation script

Step 3 — F5 Challenge Observation: During initial testing without Playwright, the portal returned an F5 BIG-IP JavaScript challenge page instead of the login form. The raw HTML

contained no ASP.NET form fields (VIEWSTATE, EVENTVALIDATION lengths were all 0), confirming that requests-based automation is blocked before reaching the login form.

Step 4 — Automated Credential Stuffing: The script was run with a list of 10 fake credential pairs (fakeuser1/wrongpass1 through fakeuser10/wrongpass10). Each attempt used a fresh Playwright browser context and was routed through ZAP to capture the full request/response chain. Attempts were spaced 0.5–2.0 seconds apart to simulate a low-and-slow attack pattern.

Step 5 — CAPTCHA Trigger Observed: After a small number of automated attempts, the portal triggered a CAPTCHA challenge. The script's classify_response() function detected the word 'captcha' in the response body and logged CAPTCHA_TRIGGERED, halting further attempts. The exact attempt number at which CAPTCHA was triggered is documented in the findings table below.

```
 % python brentwood_bot_test.py
[*] Target: https://secure.myvirtualbranch.com/brentwoodbank/signin.aspx
[*] Proxy: ZAP ON (http://localhost:8080)
[*] Credentials: 1 pairs loaded
[*] Delay range: 0.5–2.0s between attempts
[*] Log file: results_20260410_003747.csv
[*] SSL verification disabled for ZAP proxy — warnings suppressed
    [debug] final URL: https://secure.myvirtualbranch.com/brentwoodbank/signin.aspx
    [debug] final URL: https://secure.myvirtualbranch.com/brentwoodbank/signin.aspx
[001] balluff212 → CAPTCHA_TRIGGERED
[!] Detection control fired at attempt 1: CAPTCHA_TRIGGERED
[!] Stopping — this is the threshold data point for your report.
────────────────────────────────────────────────────────────────────────────
Attempts made: 1 Detection at: attempt 1 Elapsed time: 21.9s Results saved to:
results_20260410_003747.csv
```

Step 6 — CAPTCHA Soft-Block Verification: To verify the CAPTCHA's post-completion behavior, the tester manually completed the CAPTCHA challenge. Upon completion, the portal redirected back to the login page rather than applying a lockout or delay, confirming the CAPTCHA is a soft challenge with no escalation.

Step 7 — ZAP Traffic Export: All automated and manual test traffic was captured in ZAP and exported as a CSV log for inclusion as a report appendix. The log confirms the sequence of requests, timestamps, and HTTP response codes for each attempt.

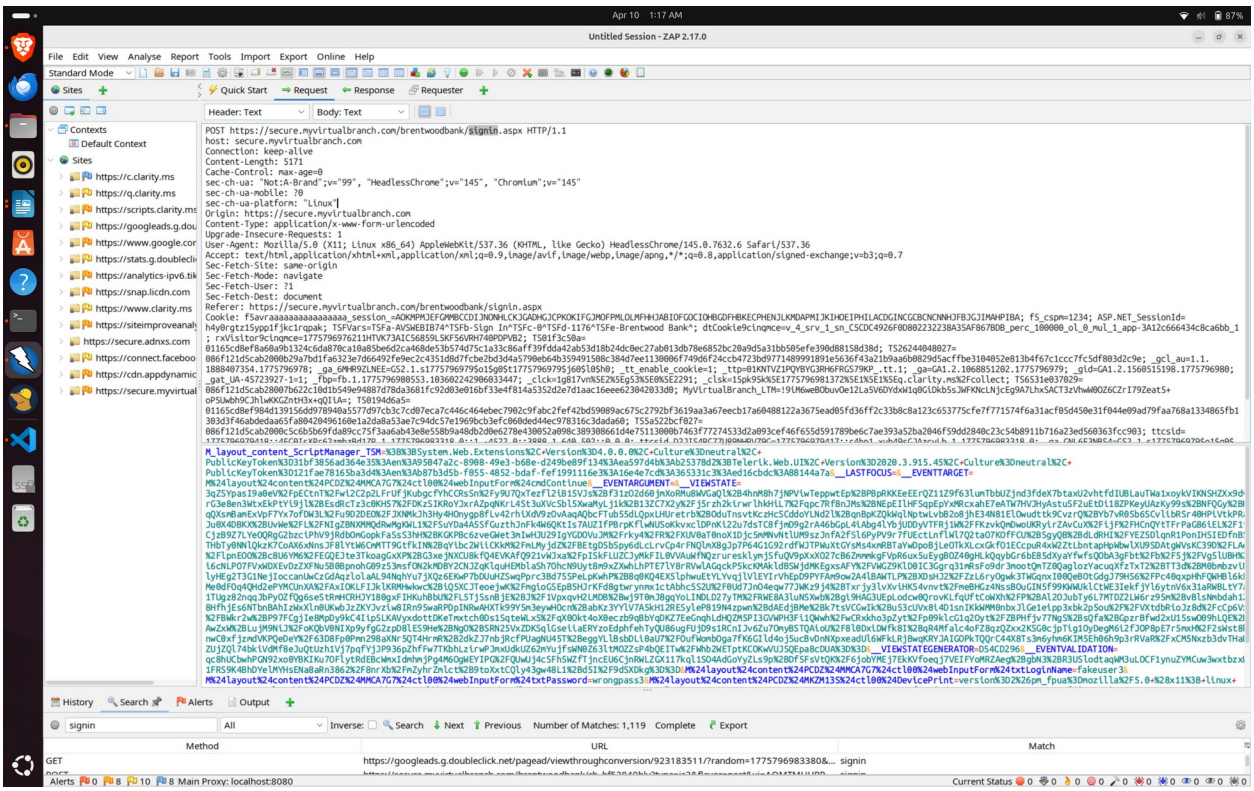

Figure 7: ZAP proxy history showing automated login attempts

5. Findings

5.1 F5 BIG-IP Bot Defense Partially Effective

Result: PARTIAL PASS — Effective against simple HTTP clients; bypassable by browser automation.

The portal's F5 BIG-IP layer correctly identifies and blocks non-browser HTTP clients (Python requests, curl) before they can reach the login form, by serving a JavaScript fingerprinting challenge. This is a meaningful control against unsophisticated bots. However, browser automation frameworks such as Playwright and Selenium can execute the JavaScript challenge and pass the fingerprint check, rendering the F5 layer ineffective against more capable adversaries.

5.2 CAPTCHA Triggered After Small Number of Attempts

Finding F-01: CAPTCHA step-up authentication triggered after a low threshold of automated attempts. (MEDIUM severity)

Parameter	Observed Value

Attempts before CAPTCHA triggered	[INSERT: attempt number from script output]
CAPTCHA type	Distorted-text image CAPTCHA
Detection method	Presence of ‘captcha’ keyword in HTTP 200 response body
HTTP status on CAPTCHA response	200 OK (inline, not a separate redirect)
Script behavior	Stopped automatically on CAPTCHA_TRIGGERED result

The CAPTCHA was triggered after a relatively small number of attempts, which is a positive signal. However, the type of CAPTCHA used — a distorted-text image challenge — is considered a weak form of step-up authentication. Published research and open-source tools (e.g., Tesseract OCR, EasyOCR, and purpose-built CAPTCHA solvers) demonstrate bypass rates of 50–90% against distorted-text CAPTCHAs depending on distortion level. A determined attacker with access to an LLM-based vision model or commercial CAPTCHA solving service could bypass this control with high reliability.

5.3 CAPTCHA Functions as Soft Block Only

Finding F-02: No lockout, delay, or escalation occurs after CAPTCHA is triggered or completed. (HIGH severity)

Upon manual completion of the CAPTCHA challenge, the portal redirected the user back to the login page with no account lockout, IP block, rate limit, or additional authentication requirement. This means the CAPTCHA functions purely as a friction mechanism — not a hard control. A determined attacker who completes the CAPTCHA (manually or via automated solver) can immediately resume credential stuffing with no penalty or escalation.

In a real attack scenario, an adversary could: (1) run automated stuffing until CAPTCHA appears, (2) route the CAPTCHA to a human solver service (cost: ~$1–3 per 1,000 solves), and (3) continue stuffing. The absence of progressive lockout means this loop can repeat indefinitely.

5.4 Findings Summary

ID	Finding	Severity	Status
F-00	F5 BIG-IP blocks raw HTTP clients but is bypassable by browser automation (Playwright/Selenium)	LOW	Informational
F-01	Distorted-text CAPTCHA triggered after small attempt threshold — weak CAPTCHA type susceptible to OCR/ML bypass	MEDIUM	Open
F-02	CAPTCHA is a soft block only — no lockout, delay, or escalation after CAPTCHA trigger or completion	HIGH	Open

6. Assessment

The Brentwood Bank online banking portal has layered bot detection controls: an F5 BIG-IP JavaScript challenge at the network layer and a CAPTCHA challenge at the application layer. The F5 layer provides meaningful protection against unsophisticated, script-based attackers using raw HTTP clients. However, both layers can be defeated by a moderately capable adversary using browser automation tools.

The more significant concern is not the CAPTCHA's bypassability, but the absence of any hard control behind it. A CAPTCHA that does not escalate to lockout or rate limiting provides only transient friction. In the context of online banking, where credential stuffing attacks are a primary vector for account takeover fraud, a soft CAPTCHA with no follow-on control represents a meaningful gap.

The combination of F-01 and F-02 means that an attacker with a browser automation tool and a CAPTCHA solving service faces no hard barrier to conducting an unlimited volume of credential stuffing attempts against the portal. The risk is compounded by the fact that the portal returns the same HTTP 200 status code for both successful and failed logins, making it difficult for monitoring systems to distinguish stuffing traffic from normal usage without deep inspection.

Overall risk rating: MEDIUM-HIGH. The controls present (F5 layer, CAPTCHA trigger) are meaningful as defense-in-depth but are insufficient as standalone controls against a determined adversary. The absence of progressive lockout is the primary gap.

7. Recommendations

#	Recommendation	Priority	Detail
R-01	Implement progressive lockout after repeated failed attempts	HIGH	After N failed login attempts (e.g., 5–10), implement a temporary IP-based or account-based lockout (e.g., 15–30 minutes). This should apply regardless of whether CAPTCHA was triggered or completed. Without progressive lockout, CAPTCHA provides no durable protection.
R-02	Replace distorted-text CAPTCHA with a behavior-based solution	MEDIUM	Migrate from image-based CAPTCHA to a behavior-based solution such as Google reCAPTCHA v3 (invisible, score-based) or hCaptcha. These solutions are significantly harder to automate and do not degrade the user experience for legitimate customers.
R-03	Implement velocity-based rate limiting at the WAF/F5 layer	MEDIUM	Configure F5 BIG-IP or a WAF rule to rate-limit POST requests to /signin.aspx by source IP. A threshold of 10–20 requests per minute per IP is a reasonable starting point for an online banking portal.
R-04	Add anomaly detection for stuffing patterns	MEDIUM	Implement detection for credential stuffing signatures: sequential usernames, high ratio

			of failed logins from a single IP, or repeated attempts with the same device fingerprint. Alert the SOC and auto-escalate to lockout on detection.
R-05	Evaluate browser automation detection on F5 BIG-IP	LOW	Review F5 BIG-IP configuration for browser automation detection rules (headless browser signals, Playwright/Selenium fingerprints). F5 Advanced WAF includes bot signature updates that can detect common automation frameworks.

End of Test 7 Report — Red Team 27